

ENCN205

Applied Data Analysis for Civil and Natural Systems

LECTURE 11

The Future of GIS - AI Coding Agents

University of Canterbury | Te Whare Wānanga o Waitaha

Dr Alberto Ardid

Lecturer / Assistant Professor
Department of Civil and Environmental Engineering
University of Canterbury, New Zealand

E319 |
alberto.ardid@canterbury.ac.nz

"What if you could describe a map in English and watch it get built?"

Who has used ChatGPT, Copilot, or similar to help write code?

Today is about a step beyond that: tools that write AND run the code themselves, in a loop, until the task is done.

You've spent 5 weeks learning to write the code yourself.

Today: what changes when an AI agent can write it for you — and what doesn't.

Today's Outline

1. From GUI to Python to AI agents — the next step in the course's own arc
2. What an AI coding agent actually is
3. LIVE DEMO: building an interactive map with Claude Code
4. Promise, pitfalls, and professional obligations
5. Other future trends in GIS (brief)
6. What still can't be automated

From Clicking to Directing



Each stage changed what a GIS analyst spends their time doing — it did not remove the need to understand GIS.

What Is an AI Coding Agent?

Autocomplete tools (e.g. classic GitHub Copilot):

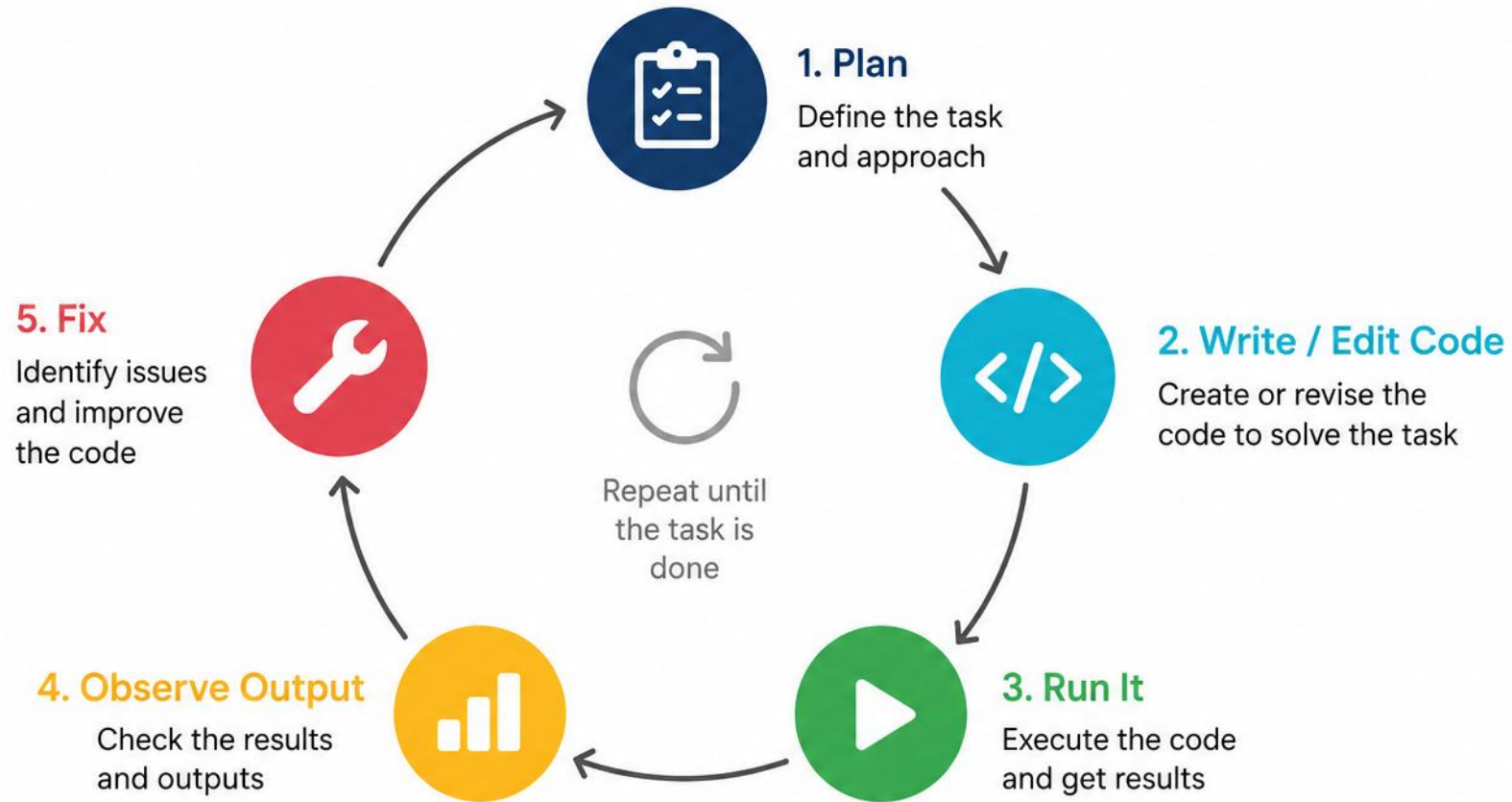
Suggest the next line or block as you type. You still drive.

Agentic tools (e.g. Claude Code, Cursor's agent mode, Codex):

Given a goal in plain English, they read your files, write code, RUN it, read the output or errors, and keep going until the task is done - with little or no line-by-line typing from you.

The key difference: agentic tools close the loop themselves.

The Agentic Loop



This is the same debugging cycle you've used by hand all course — just automated, and much faster.

Same Task, Three Ways

Task: clip building footprints to a flood-zone buffer, colour by exposure, export a map.

By hand in a GUI:

Open QGIS, load layers, buffer tool, clip tool, style manually, export — ~30-45 minutes, not reproducible next time.

Written yourself in Python (this course):

`gpd.read_file`, `.buffer()`, `gpd.clip()`, `matplotlib` styling — ~15-20 minutes once you know the functions, fully reproducible.

Directed with an AI agent:

Describe the task in one or two sentences; the agent writes and runs the GeoPandas code, checks the output, fixes errors — a few minutes, IF you specify the task clearly and check the result.

The task specification and the final check are still your job.

What a Prompt Actually Looks Like

```
> Clip buildings.gpkg to a 500m buffer around the fault line, colour by distance, save as a PNG map with a legend.
```

The agent:

Opens the files to check column names and CRS

Writes a GeoPandas script and runs it

Notices the CRS is in degrees, reprojects to NZTM, re-runs

Shows you the resulting map

Notice: it caught the CRS mistake itself — the same mistake Lecture 4 spent a whole lecture on.

Always ask: 'did it use the right CRS and data?'

Building a real, interactive web map of New Brighton flood exposure - live, from a blank page.

Data: real Christchurch building footprints, coastline, roads, and modelled flood zones - the same New Brighton data from Lecture 1's layer-toggle demo.

Goal: an interactive map with a sea-level slider that live-recolours exposed buildings.

If something breaks: that's not a failure - that's the loop from a few slides ago, happening in real time.

LIVE DEMO

Building an Interactive Map, Live, with Claude Code

(reference build: [Demos/L11_ClaudeCodeMapDemo.html](#))

Scan to open the demo on your phone



aardid.github.io/uc_205_gis_course/_L11_ClaudeCodeMapDemo.html

What Just Happened?

The agent read real data files to understand their structure before writing any code.

It wrote a complete, working web page - HTML, CSS, and JavaScript - not just a snippet.

It ran and checked its own work, and fixed problems without being told exactly how.

Every step chained several tool calls together automatically - reading, writing, running, checking.

It did not 'understand' New Brighton, flood risk, or GIS - it followed clear instructions and used the data and framing YOU gave it.

Promise and Pitfalls

Promise

Genuinely fast for well-specified tasks

Lowers the barrier to building things (maps, dashboards, small tools)

Can catch some classic mistakes by actually running the code (like CRS errors)

Pitfalls

No domain judgment - doesn't know if 'exposed' should mean a 100m or 500m buffer unless you say so

Can write code that runs without errors but is still wrong (wrong join, double-counted area, mislabelled legend)

Confidently wrong looks identical to confidently right until you check

This is the exact same failure shape as Lecture 2's 'Python won't warn you' and Lecture 4's CRS mismatch — just one level up the stack.

Always ask: 'how do I know this is right?'

Professional Obligations

You are still the engineer of record - the agent is not accountable, you are.

Verify outputs the same way you would verify a junior colleague's work: check the method, not just whether it 'looks right'.

Document AI use - this course already requires an AI declaration on your assignment; treat that as training for real practice.

Engineering NZ's Code of Ethical Conduct (Lecture 2) applies to AI-assisted work exactly as it applies to any other tool.

Other Trends Worth Knowing

GeoAI / machine learning - automated feature extraction from imagery, predictive hazard mapping (Lecture 10 touched on this)

Digital twins - live virtual replicas of cities and infrastructure for scenario testing

Real-time sensors and IoT - continuous monitoring feeding directly into GIS

Drones, photogrammetry, and LiDAR - cheaper, faster ground-truth data collection

Each of these is its own field. Today's focus was deliberately narrower: the tools you use to WRITE and RUN analysis, not just the data feeding into it.

What Still Can't Be Automated

Knowing which CRS is right for the job

Judging whether data is fit for purpose

Choosing what 'exposed' or 'at risk' should mean for a specific project

Deciding whether a result is trustworthy enough to put your name on

"AI enhances GIS - it does not replace spatial thinking."

Today's Takeaways

1. Agentic AI tools close the write-run-fix loop themselves, not just suggest the next line
2. This is the same debugging loop you've used all course, automated and sped up
3. Speed and confidence are not the same as correctness — verify outputs like a colleague's work
4. You remain professionally accountable for AI-assisted work, and must document its use
5. The fundamentals from Lectures 1-10 are what let you use these tools well

Course Wrap-Up

What You Have Learned

GIS foundations, data quality, vector analysis, cartography

Raster analysis, terrain products, hydrological modelling

Risk assessment, MCE, spatial decision-making

And now: how AI coding agents are changing the way this work gets done

All implemented in Python - reproducible, scalable, industry-ready

Exam Preparation: Study Tool (347 MCQs) | Practice Questions | Office Hours

Written Exam: 2 hours | 6 questions × 10 marks | 30% of course grade (min. 40% to pass)

Discussion

**Pick a task from one of your labs. How would you do it
(a) by hand, (b) as a Python script, (c) by directing an AI agent?**

Consider:

A buffer + spatial join (Lab 2), a CRS reprojection (Lab 1), or a raster classification (Lab 3)

What changes at each step, and what stays the same?

What could go wrong with (c) that wouldn't happen with (b)?

Try It Yourself

Claude Code and similar tools are available to students - check current free/student access.

Start small: ask it to explain or refactor code you already wrote for a lab, not to write something from scratch.

Course policy reminder: AI use is permitted only for Python programming/debugging assistance, and must be declared - the same rule applies whether it's autocomplete or a full agent.

Best habit to build now: always read and understand what it wrote before you run it.

Where This Leads

300-level: ENCN3xx courses building on GIS and data analysis

Honours / research: GIS + AI is an active research area right now

Industry: NZ consultancies are already adopting both Python workflows and AI coding tools

The skill that's growing in value: directing these tools well — clear task specification, and knowing how to check the output